

AWS DevOps Hands-On Project

Linux Administrator → DevOps Engineer

Project: Enterprise Web Application DevOps Platform on AWS

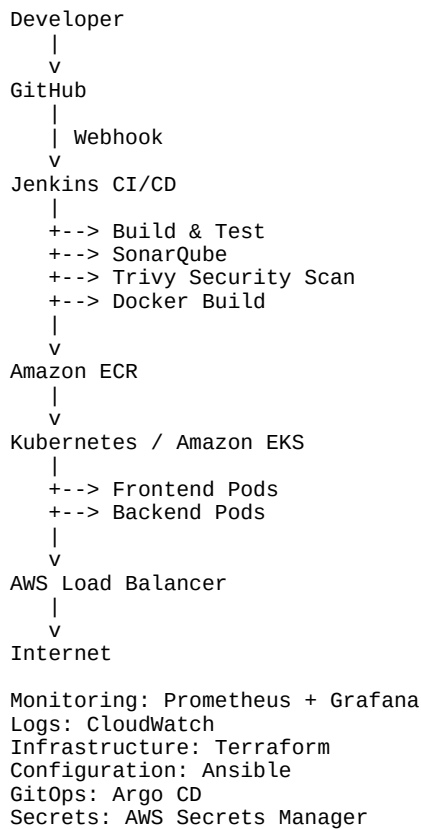
A step-by-step practical roadmap covering Git, GitHub, Docker, Jenkins, Kubernetes, Amazon EKS, ECR, Terraform, Ansible, Helm, Trivy, SonarQube, Prometheus, Grafana, CloudWatch, AWS Secrets Manager and Argo CD.

Learning approach: Build one realistic project progressively instead of learning DevOps tools in isolation.

1. Project Objective

The goal is to build, automate, deploy, monitor and secure a small web application using a production-style DevOps workflow. Your existing Linux administration knowledge will be used as the foundation.

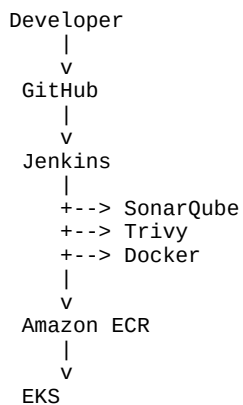
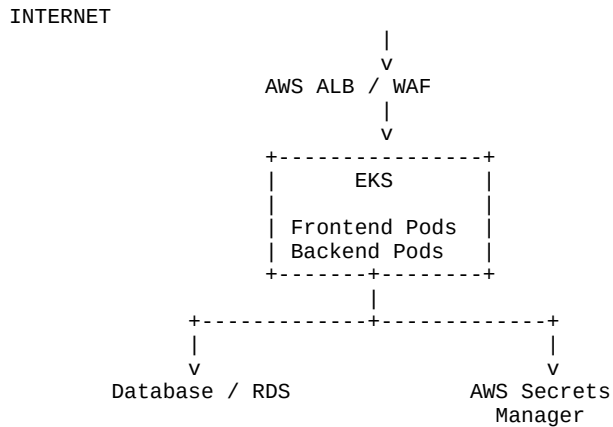
The final workflow will be:



2. Tools Covered

Area	Tool
Operating system / administration	Linux, Amazon Linux
Source control	Git, GitHub
CI/CD	Jenkins
Containers	Docker
Container registry	Amazon ECR
Orchestration	Kubernetes, Amazon EKS
Packaging	Helm
Infrastructure as Code	Terraform
Configuration management	Ansible
Security	Trivy
Code quality	SonarQube
Monitoring	Prometheus, Grafana
Logging	Amazon CloudWatch
Secrets	AWS Secrets Manager
GitOps	Argo CD
AWS CLI / cluster tools	AWS CLI, kubectl, eksctl

3. Final AWS Architecture



Terraform provisions infrastructure.
 Ansible configures Linux systems.
 Prometheus + Grafana monitor workloads.
 Argo CD provides GitOps deployment.
 CloudWatch provides AWS/application logs.

4. Project Repository Structure

```

devops-shop/
|--
+-- frontend/
|   +-- index.html
|   +-- Dockerfile
|
+-- backend/
|   +-- app.py
|   +-- requirements.txt
|   +-- Dockerfile
|
+-- docker/
|   +-- docker-compose.yml
|
+-- kubernetes/
|   +-- namespace.yaml
|   +-- deployment.yaml
|   +-- service.yaml
|   +-- configmap.yaml
|   +-- secret.yaml
|   +-- ingress.yaml
|
+-- helm/
|   +-- devops-shop/
|
+-- terraform/
|   +-- provider.tf
|   +-- variables.tf
|   +-- vpc.tf
|   +-- eks.tf
|   +-- ecr.tf
|   +-- iam.tf
|   +-- outputs.tf
|   +-- terraform.tfvars
  
```

```
|
+-- ansible/
|   +-- inventory
|   +-- playbook.yml
|
+-- Jenkinsfile
+-- README.md
```

5. Learning Roadmap

Stage 1 — DevOps foundation

Git, GitHub, AWS CLI, SSH and the AWS lab environment.

Stage 2 — Application

Create a small frontend and backend API. Software development is intentionally simple; the focus is DevOps.

Stage 3 — Docker

Dockerfiles, images, containers, networks, volumes, Compose and image optimization.

Stage 4 — Jenkins

Install Jenkins, connect GitHub, create pipelines, webhooks, build/test/scan/package stages.

Stage 5 — Kubernetes

Pods, Deployments, ReplicaSets, Services, ConfigMaps, Secrets, Ingress, storage, Jobs and HPA.

Stage 6 — Amazon EKS

Create an AWS Kubernetes cluster, worker nodes, networking, IAM, load balancing and application deployment.

Stage 7 — Terraform

Provision VPC, subnets, security groups, IAM, ECR and EKS as Infrastructure as Code.

Stage 8 — Ansible

Automate Linux configuration, packages, services, users and system configuration.

Stage 9 — Monitoring

Prometheus metrics, Grafana dashboards, alerts, pod health and troubleshooting.

Stage 10 — Advanced DevOps

Helm, Argo CD, GitOps, secrets, production-style CI/CD and operational troubleshooting.

6. Stage 1 — AWS Lab Setup

Start small. Do not create the complete production architecture on day one because EKS, NAT Gateways, load balancers, RDS and other AWS resources can incur charges.

Recommended initial lab

```
OS: Amazon Linux
Purpose: initial Jenkins / Docker / DevOps lab
Access: SSH
Tools introduced progressively:
AWS CLI
Git
Docker
Java
Jenkins
kubectl
eksctl
Terraform
Helm
```

Initial verification commands

```
cat /etc/os-release
uname -r
whoami
hostname
df -h
free -h
ip addr
ip route
aws --version
git --version
docker --version
java -version
```

Important: We will install and verify each tool one at a time. Avoid installing everything with an unverified script.

7. Git and GitHub

Learning objectives

Understand repositories, branches, commits, remotes, pull requests, tags and basic branching strategy.

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"

mkdir devops-shop
cd devops-shop
git init

git status
git add .
git commit -m "Initial commit"

git branch
git remote -v
git push
```

Later, GitHub will trigger Jenkins through a webhook whenever code is pushed.

8. Stage 2 — Application

Create a deliberately simple application so that most of the learning time goes into the DevOps lifecycle.

Suggested application

```
Frontend: Nginx serving a simple web page
Backend: Python Flask REST API
Database: introduced later, preferably managed through AWS when we reach the advanced phase
```

Example API endpoints can include /health and /api/products. The application should expose a health endpoint so Kubernetes probes and monitoring can test it.

9. Stage 3 — Docker

Core concepts

Learn the difference between an image and a container, container lifecycle, ports, networks, volumes, Dockerfiles, registries and image tags.

```
docker build -t devops-shop:1.0 .
docker images
docker run -d --name devops-shop -p 8080:8080 devops-shop:1.0
docker ps
docker logs devops-shop
docker exec -it devops-shop /bin/sh
docker stop devops-shop
docker rm devops-shop
```

Target flow

```
Application
|
v
Dockerfile
|
v
Docker image
|
v
Amazon ECR
```

10. Stage 4 — Jenkins CI/CD

Install Jenkins initially on the lab EC2 instance. Later, the architecture can be changed to use Kubernetes-based Jenkins agents.

Pipeline stages

```
Checkout
|
Build
|
Unit Test
|
SonarQube
|
Trivy Scan
|
Docker Build
|
Push Image to ECR
|
Deploy to EKS
|
Smoke Test
```

Starter Jenkinsfile

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build') {
            steps {
                sh 'docker build -t devops-shop:${BUILD_NUMBER} .'
            }
        }
    }
}
```

```

    stage('Test') {
        steps {
            sh 'echo "Run application tests here"'
        }
    }

    stage('Security Scan') {
        steps {
            sh 'trivy image devops-shop:${BUILD_NUMBER}'
        }
    }

    stage('Push Image') {
        steps {
            sh './push-to-ecr.sh'
        }
    }

    stage('Deploy') {
        steps {
            sh 'kubectl apply -f kubernetes/'
        }
    }
}

```

We will build this pipeline incrementally rather than copying a final pipeline and treating it as a black box.

11. Stage 5 — Kubernetes

Objects to master

```

Namespace
Pod
Deployment
ReplicaSet
Service
ConfigMap
Secret
Ingress
PersistentVolume
PersistentVolumeClaim
StatefulSet
DaemonSet
Job
CronJob
HorizontalPodAutoscaler

```

Essential troubleshooting commands

```

kubectl get nodes
kubectl get pods -A
kubectl get svc
kubectl get deployments
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- /bin/sh
kubectl get events --sort-by=.lastTimestamp
kubectl rollout status deployment/<deployment>
kubectl rollout history deployment/<deployment>

```

Important failure states

Learn to diagnose CrashLoopBackOff, ImagePullBackOff, Pending, OOMKilled, FailedScheduling, readiness/liveness probe failures and service connectivity problems.

12. Stage 6 — Amazon EKS

Move the Kubernetes application from a local lab cluster to Amazon EKS. Learn AWS networking, IAM, worker nodes, Kubernetes networking and AWS load balancing.

Target

```

AWS VPC
|

```

```

+-- Public/Private Subnets
|
+-- IAM
|
+-- EKS
    |
    +-- Worker Nodes
        |
        +-- Frontend Pods
        +-- Backend Pods

```

Basic EKS verification

```

aws sts get-caller-identity
aws eks list-clusters
aws eks update-kubeconfig --region <region> --name <cluster-name>
kubectl get nodes
kubectl get pods -A

```

13. Stage 7 — Terraform

Terraform becomes the source of truth for infrastructure. The objective is to stop manually creating infrastructure through the AWS Console.

Terraform structure

```

terraform/
+-- provider.tf
+-- variables.tf
+-- vpc.tf
+-- eks.tf
+-- ecr.tf
+-- iam.tf
+-- outputs.tf
+-- terraform.tfvars

```

Core commands

```

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
terraform output
terraform state list
terraform destroy

```

Learn Terraform state, variables, outputs, modules, remote state and safe change management before using it for a production-style EKS environment.

14. Stage 8 — Ansible

Use Ansible to automate Linux configuration. This is where your Linux administrator background becomes a major advantage.

```

- name: Configure Docker servers
  hosts: docker_servers
  become: true

  tasks:
    - name: Install Docker
      package:
        name: docker
        state: present

    - name: Enable and start Docker
      service:
        name: docker
        state: started
        enabled: true

```

Practice inventories, variables, handlers, templates, roles, idempotency and secrets handling.

15. Stage 9 — Security

Trivy

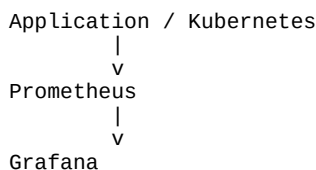
Use Trivy to scan container images for known vulnerabilities before pushing or deploying them.

```
trivy image devops-shop:1.0
```

SonarQube

Use SonarQube for static analysis and code-quality gates. The Jenkins pipeline should fail when agreed quality/security thresholds are not met.

16. Stage 10 — Monitoring and Logging



Build dashboards for CPU, memory, pod count, restarts, application availability and selected application metrics. Use CloudWatch for AWS and infrastructure logs where appropriate.

Operational exercise

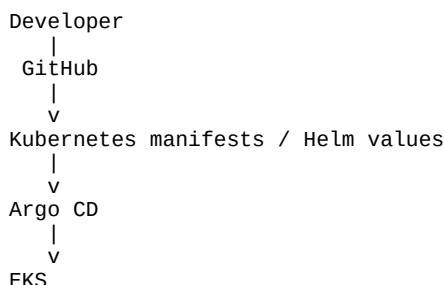
Intentionally break a deployment, then investigate it using kubectl, application logs, Kubernetes events, metrics and AWS console/CLI information. The objective is to develop real troubleshooting skill.

17. Stage 11 — Helm

Convert raw Kubernetes manifests into a reusable Helm chart.

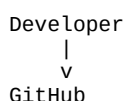
```
helm create devops-shop
helm lint ./devops-shop
helm template devops-shop ./devops-shop
helm install devops-shop ./devops-shop
helm list
helm upgrade devops-shop ./devops-shop
helm rollback devops-shop <revision>
```

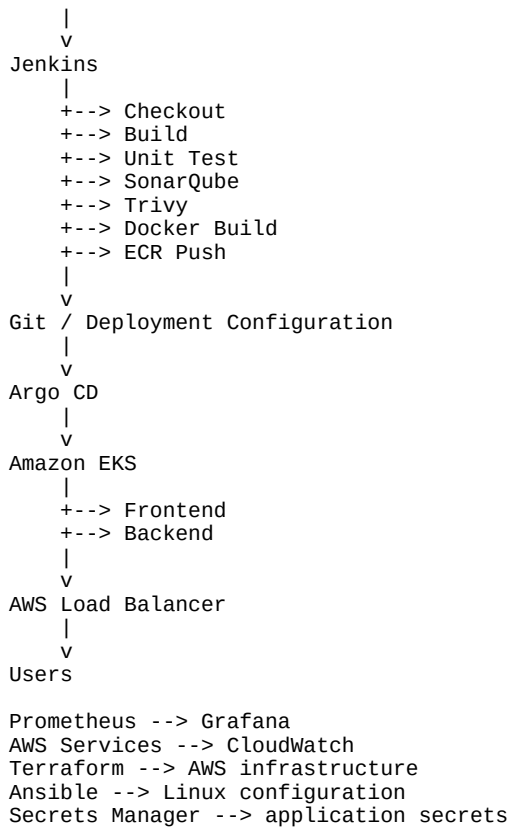
18. Stage 12 — Argo CD / GitOps



The key idea is that Git becomes the desired-state source. Argo CD continuously reconciles the Kubernetes environment with the declared state in Git.

19. Final CI/CD Flow





20. Troubleshooting Checklist

- Linux: CPU, memory, disk, processes, services, network, DNS, ports and logs.
- Docker: image build errors, container exit codes, permissions, ports, networks and volumes.
- Jenkins: agent connectivity, credentials, workspace, pipeline syntax, Docker access and failed stages.
- ECR: AWS identity, repository name, image tags and registry authentication.
- Kubernetes: events, pod status, image pull, probes, services, DNS, resources and scheduling.
- EKS: kubeconfig, IAM permissions, node status, networking, security groups and AWS load balancer integration.
- Terraform: state, drift, dependency changes, plan output and safe destroy/apply workflow.
- Ansible: inventory, SSH connectivity, privilege escalation, idempotency and failed tasks.
- Monitoring: missing metrics, targets, dashboards, alerts and resource saturation.

21. Interview Preparation

At the end of each stage, practice explaining the technology without memorized definitions.

```

Linux:
  How do you troubleshoot high CPU?
  How do you troubleshoot a full filesystem?
  How do you diagnose a port connectivity problem?

Git:
  merge vs rebase?
  branch strategy?
  revert vs reset?

Docker:
  image vs container?
  Dockerfile best practices?
  container networking?

```

Jenkins:
 CI vs CD?
 Pipeline vs Freestyle job?
 Jenkinsfile?
 Credentials?
 Agents?

Kubernetes:
 Pod vs Deployment?
 Service vs Ingress?
 ConfigMap vs Secret?
 Readiness vs Liveness?
 CrashLoopBackOff troubleshooting?

Terraform:
 State?
 Plan vs Apply?
 Module?
 Drift?
 Remote state?

Ansible:
 Idempotency?
 Inventory?
 Role?
 Handler?
 Terraform vs Ansible?

AWS/EKS:
 VPC?
 IAM?
 EKS?
 ALB?
 ECR?
 Private vs public subnet?

22. Cost and Cleanup Strategy

Do not leave expensive AWS resources running unnecessarily. After each lab, identify what was created and remove resources that are no longer needed.

```
terraform plan
terraform destroy

aws eks list-clusters
aws ecr describe-repositories
aws ec2 describe-instances
```

Use AWS Budgets/Cost monitoring for the lab account and keep the architecture minimal until each component is understood.

23. Final CV Project Statement

Designed and implemented an end-to-end AWS DevOps CI/CD platform using GitHub, Jenkins, Docker, Amazon ECR, Kubernetes/EKS, Terraform, Ansible, Helm, Trivy, Prometheus, Grafana and Argo CD, implementing automated build, security scanning, container image delivery, Kubernetes deployment, infrastructure provisioning, monitoring and GitOps workflows.

24. Recommended Teaching Sequence

We should execute this project interactively. Each lesson should contain: concept → architecture → exact commands → verification → expected output → troubleshooting → cleanup → interview questions.

```
Lesson 1 : AWS lab + Linux preparation
Lesson 2 : Git + GitHub
Lesson 3 : Application creation
Lesson 4 : Docker
Lesson 5 : Docker Compose
Lesson 6 : Jenkins installation
Lesson 7 : First Jenkins pipeline
Lesson 8 : Jenkins + Docker + ECR
Lesson 9 : Kubernetes fundamentals
Lesson 10 : Kubernetes application deployment
```

Lesson 11 : Terraform fundamentals
Lesson 12 : Terraform VPC
Lesson 13 : Terraform EKS + ECR
Lesson 14 : EKS application deployment
Lesson 15 : Ansible
Lesson 16 : Helm
Lesson 17 : Trivy + SonarQube
Lesson 18 : Prometheus + Grafana
Lesson 19 : CloudWatch + troubleshooting
Lesson 20 : Argo CD / GitOps
Lesson 21 : End-to-end CI/CD
Lesson 22 : Production-style troubleshooting
Lesson 23 : Cost optimization and cleanup
Lesson 24 : Interview / CV preparation

25. First Practical Exercise

Start with only the AWS lab instance. Do not install the entire toolchain at once. The next hands-on lesson should create the EC2 instance, connect through SSH, verify the Linux environment, configure AWS CLI safely, install Git and Docker, and verify each step before proceeding.

This staged approach will let you understand how the tools fit together and will make troubleshooting much easier than following a large copy/paste installation script.